Universität Zürich UZH

GEO877

# Spatial Algorithm

## 03. Segment and intersection

Inhye Kong

# Feedback/solution for your practical 2

- Practical 2 was about PointGroup and Polygon, and you are expected to implement area and centroid

- To add the code to calculate area and centroid in the existing Polygon class
- To compare the centroid (i.e. area-weighted) and mean point on the map. Are they different? Why?

- Update your .py package (you just need to call them later whenever in needed)

Recap

# Basic elements of (spatial) data

- **Point** in coordinates (x,y) assuming Cartesian space
- **PointGroup** as a collection of points
- **Segment** as a finite line with a start and an end point
  - Segments are bounded at both ends
  - Rays have a start but no end
  - Lines have neither a start nor an end
- **Polygon** with an ordered set of point coordinates

- We will not cover raster, but can be implemented for your own code if necessary

# Line

Let's start with how to mathematically express a line – the infinite one

$$a_1x + b_1y + c_1 = 0 \qquad \text{or} \qquad y = \frac{a_1}{b_1}x + \frac{c_1}{b_1}$$

Given any two points on a line, you can derive a, b, and c

# Line intersection

Then, we can test if two lines will intersect:

Two lines $l_1$ and $l_2$ will intersect if the determinant is non-zero, e.g.,

$$a_1x + b_1y + c_1 = 0$$
$$a_2x + b_2y + c_2 = 0$$

$$\begin{vmatrix} a_1 & b_1 \\ a_2 & b_2 \end{vmatrix} = a_1b_2 - a_2b_1 <> 0$$

Imagine, there exists k, which works like: $ka_1 = a_2$; $kb_1 = b_2$; $kc_1 = c_2$

Then, $a_1b_2 - a_2b_1 = 0$, i.e., collinear
i.e. they do not intersect

But is this an interesting spatial question?

# Segment

- A finite bounded direct line between two points

- Property:
  - It has a start and an end point so that you can measure its **length**
  - It can have a direction (i.e. directed edge) very useful characteristics for rays that we will handle later

- Behaviour:
  - Equality: we can test if two (or more) segments are the same (i.e. if they share the same start and end points)
  - Connectivity: we can test if the end of a segment is a start of another segment
  - **Intersection**: we can find out if a segment/line crosses another segment/line

# Segment intersection: Bbox

**Using a bounding box approach** – the case of a pair of segments (1:1)

How did you generate a bounding box from a PointGroup? How can this work for testing a segment intersection?

Creating a Bounding Box is useful for many spatial calculations: if the bounding boxes of two geometries do not intersect, then the geometries themselves do not intersect

How efficient can this be, compared to the brute force approach? O(1)

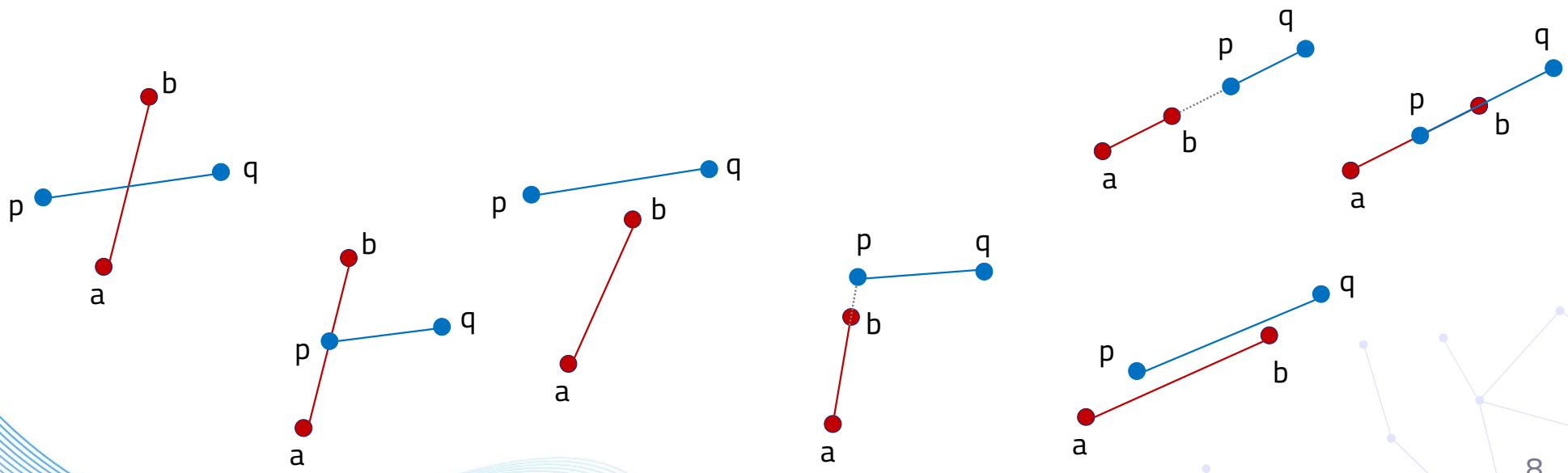
Will this approach result in correct (expected) results all the time?

- A negative result is definitive: no bbox overlap → no intersection, **always**
- A positive result is inconclusive: bbox overlap → **maybe** intersection

# Segment intersection: Bbox

**Using a bounding box approach** – the case of a pair of segments (1:1)

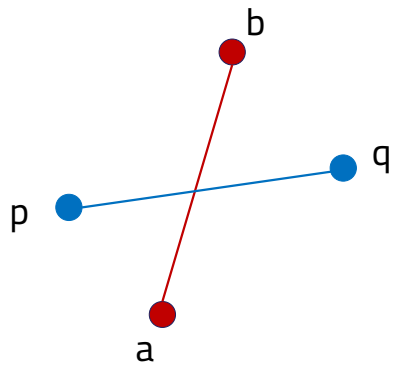
- A negative result is definitive: no bbox overlap → no intersection, **always**
- A positive result is inconclusive: bbox overlap → intersection **may not happen**



# Segment intersection: turn test

**Using a turn test** – the case of a pair of segments (1:1)

- Turn test: assume we have two segments, ab and pq. We assume a direction for a segment ab, and then which direction will this b make for p or q, respectively? Likewise for a segment pq, which direction will this q make for a and b, respectively?
- How can a turn test be a relevant solution to solve the "intersection" question?

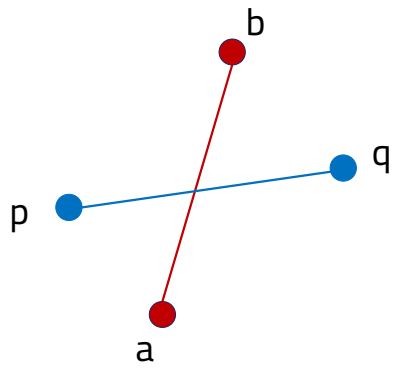


# Segment intersection: turn test

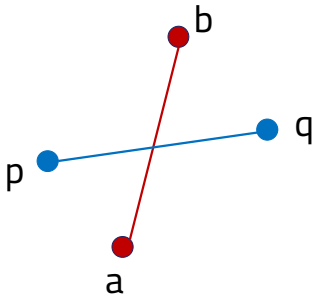
- If we have two segments **ab** and **pq**, they will intersect if:
  - **pqa** and **pqb** have different orientations (left/right) **AND**
    **abp** and **abq** have different orientations (left/right)

or:

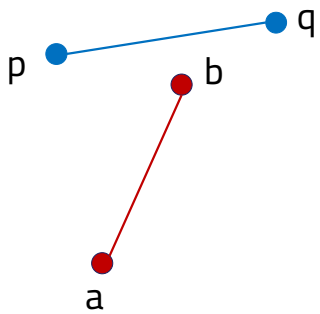
  - All the points are collinear (turn test = 0) **AND** their x and y projections overlap (bounding boxes intersect)



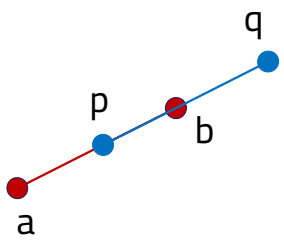
# Segment intersection: turn test



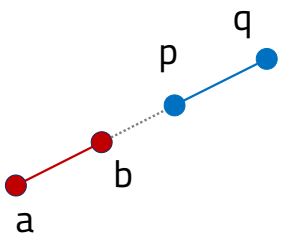

abp Left
abq Right
***and***
pqa Right
pqb Left
= intersection




abp Left
abq Right
***but***
pqa Right
pqb Right
= no intersection




Points are collinear
***and***
share the same x
and y projections
= intersection



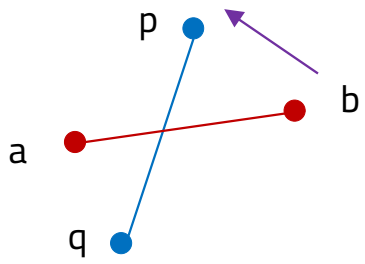

Points are collinear
***but*** do not share the
same x and y
projections
= no intersection

# Segment intersection: turn test

How to express the turn test into an algorithm? For the case of abp:

Two vectors $\overrightarrow{ab}$ and $\overrightarrow{bp}$ can be put as follows:

$\overrightarrow{ab}$ is $(b_x - a_x, b_y - a_y)$ and $\overrightarrow{bp}$ is $(p_x - b_x, p_y - b_y)$.

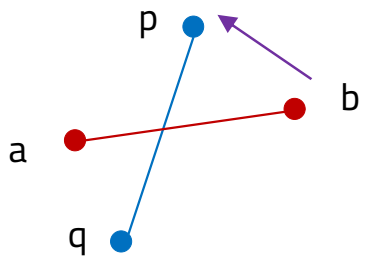



Then the 2D cross product of two vectors (i.e. determinant) is the "area of a triangle formed from three points, and this can be translated into the "orientation of the turn at point b".

$$\Delta abp = (b_x - a_x)(p_y - b_y) - (b_y - a_y)(p_x - b_x)$$

# Segment intersection: turn test

$$\Delta abp = (b_x - a_x)(p_y - b_y) - (b_y - a_y)(p_x - b_x)$$



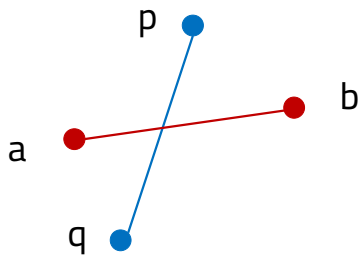

Or,

$$\Delta abp = \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ p_x & p_y & 1 \end{vmatrix} = a_x b_y - b_x a_y + b_x p_y - p_x b_y + a_x p_y - p_x a_y$$

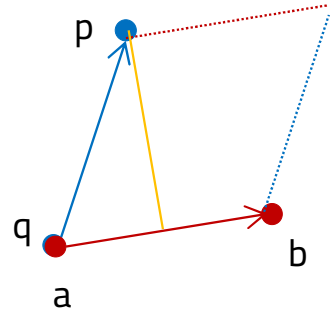
| $\Delta > 0$ | $\Delta = 0$ | $\Delta < 0$ |
|---|---|---|
| Counter-Clockwise (Left Turn) | Collinear | Clockwise (Right Turn) |

# Make (more) sense of 2D cross product (1)

- Let's imagine we have a parallelogram, made from two vectors (out of segments, i.e. $\overrightarrow{ab}$ and $\overrightarrow{pq}$ ). As we discussed, the area A is the cross product of these two vectors; here A is "signed" area, i.e. it can be both negative and positive.

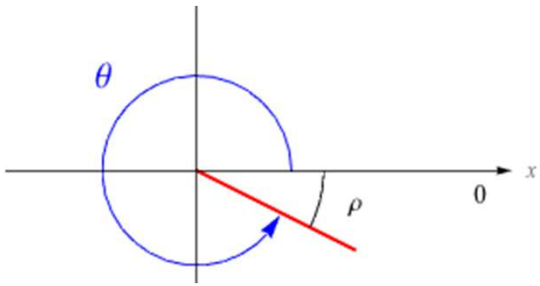


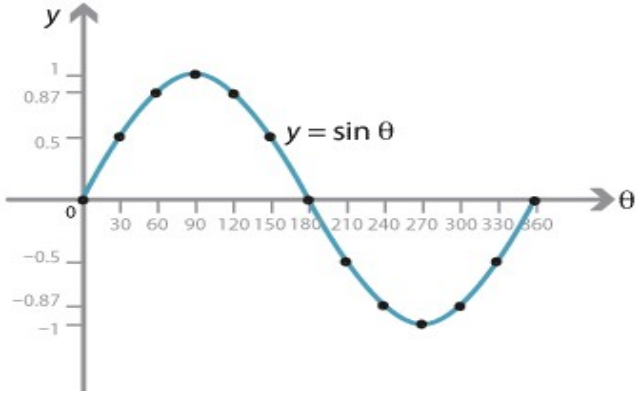

- Then, you know there is another way to calculate the area A: **parallelogram** calculation. Without the height information, the equation is put like this:
- Here, $\theta$ in $\sin\theta$ is the angle between two segments
- And $|\overline{pq}|\sin\theta$ is the height
- Signed area $A = |\overline{ab}||\overline{pq}|\sin\theta$

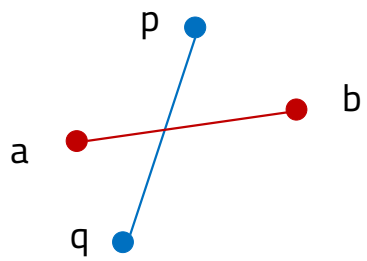


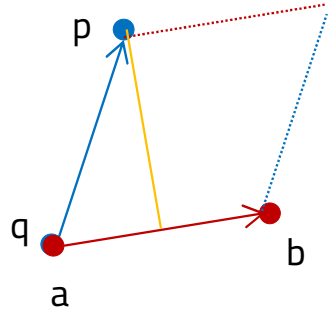
# Make (more) sense of 2D cross product (2)

- Here, $A = |\overline{ab}||\overline{pq}|\sin\theta$, the key is $\boldsymbol{\sin\theta}$ , because it is the only section that can make signs
- When you keep in mind that the angles always should be counter-clockwise, and when you recall the graph of $\sin\theta$,



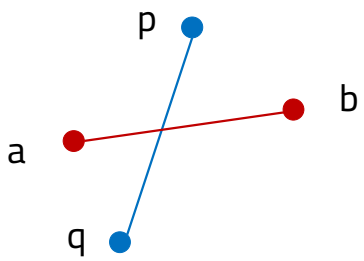



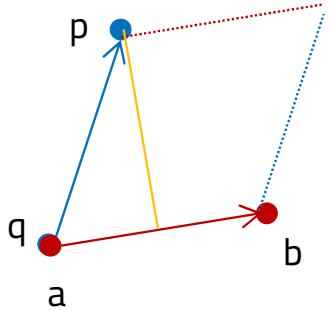



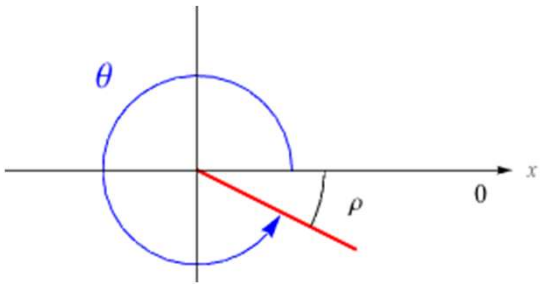



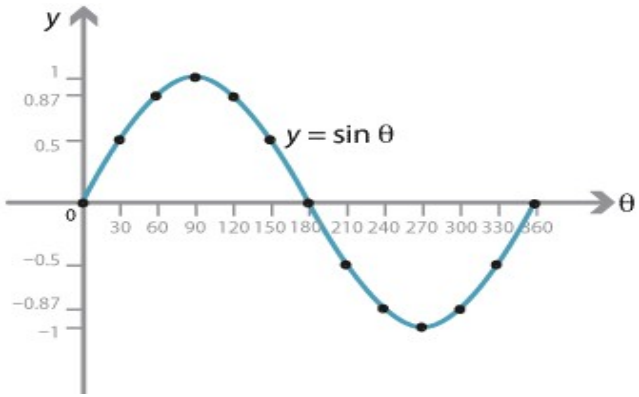
# Example

On my $\overrightarrow{ab}$ , I want to how which turn I should make to find p. When my point of interest p is on the left side of the segment $\overrightarrow{ab}$ , the angle is between 0 and 180. On this $\boldsymbol{sin\,\theta}$ plot, the angles between 0 and 180 is positive



# Practical exercise 3:

- You will be given a sample data of point pairs and play with segments and their intersections using both bounding box method and turn test method

- By hand, on the paper, you can first try to think how this data look like on coordinates
- Then design an algorithm to test intersection for this data

➢ As always, we will discuss your solutions at the beginning of next class